Reimbursement Portal

Submission Guidelines (Mandatory)

Failure to follow these guidelines will result in an immediate request for resubmission.

1. Repository Setup
    - Use your existing repository.
    - Create a new folder named **Capstone_ProjectName.**
    - Ensure the repository is set to Public so the training team can access it.
2. Branching Strategy
    - Create two separate branches:
        - Main branch: Used for raising Pull Requests (PR against main).
        - Develop branch: Used for pushing daily changes.
3. Project Organization
    - Inside the capstone folder, maintain the following structure (as screenshot below):
        - /backend
        - /frontend
4. Pull Request Process
    - Create a PR from the develop branch to the main branch.
    - Push updates to the PR daily so buddies can review and provide feedback.
5. Review & Feedback
    - Regularly check for review comments from buddies.
    - Address and resolve all feedback promptly within the PR.
6. Final Deliverables
    - An architectural walkthrough (Doc/PPT) detailing the product's system design, core components, the specific technical strategy employed during development and screenshots form UI screens
    - Product as defined in the Specifications Document

1. Introduction

1.1 Purpose

This system is a backend application that allows employees to submit reimbursement claims and enables managers or administrators to review and take action.

The system provides a structured workflow for claim processing, ensuring proper validation, tracking, and role-based access control.

The system is intended to demonstrate:

- Application of object-oriented principles
- RESTful API design using Spring Boot
- Data persistence using JPA
- Role-based workflow management
- Validation, exception handling, and testing practices

1.2 Scope

The system will support:

- User management with roles (Admin, Manager, Employee)
- Submission of reimbursement claims
- Approval or rejection of claims
- Tracking of claim status

To ensure completion within **10 days**, the following constraints apply:

- Only **basic authentication**
- **Single-level approval** (no multi-level flow)
- **Minimal UI**
- **No notifications or audit history**

1.3 Technologies

● Frontend: Html,CSS,JS (No react or other frameworks)
● Backend: Java 17, Spring Boot
● Database: PostgreSQL (or equivalent relational DB)
● ORM: JPA (Hibernate)
● Build Tool: Maven

- Testing: JUnit, Mockito
- Logging: SLF4J with Logback
- Version Control: GitHub

1.4 Definitions

- **Claim**: Request submitted by employee for reimbursement
- **Manager**: Reviews and takes action on claims
- **Admin**: Manages users and acts as fallback approver

---

2. System Overview

2.1 System Flow

Employee submits claim → System assigns reviewer → Manager/Admin reviews → Decision taken → Employee tracks status

---

2.2 User Roles

Admin

- Create and manage users
- Assign managers to employees
- Acts as fallback approver

Manager

- View assigned claims
- Approve or reject claims
- Add comments

Employee

- Submit claims
- View claim status

2.3 Assumptions & Constraints

- Each employee has **at most one manager**
- A manager can handle **multiple employees**
- If no manager is assigned → **Admin handles approval**
- Project duration: **10 days**

---

3. Functional Requirements

3.1 User Management

The system shall:

- Allow creation of users with roles (Admin, Manager, Employee)
- Allow registration only with **valid company email**
- Password Encryption needs to be there.
- Restrict emails to a predefined domain (e.g., `@company.com`)
- Allow Admin to:
    - Assign a manager to an employee
    - Remove users

3.2 Claim Management

The system shall:

- Allow employees to submit claims
- Each claim shall include:
    - Amount
    - Date
    - Description
- The system shall validate:
    - All mandatory fields are present
    - Amount is a valid positive number and should be within the limit

---

3.3 Approval Workflow

The system shall:

- Route claims to assigned manager
- Route to Admin if no manager is assigned

Reviewer (Manager/Admin) shall be able to:

- View claims
- Approve or reject claims
- Add comments

Claim statuses:

- **SUBMITTED**
- **APPROVED**
- **REJECTED**

3.4 Claim Tracking

The system shall:

- Allow employees to view submitted claims
- Display:
    - Claim status
    - Reviewer comments

3.5 Pagination

The system shall:

- Provide paginated responses for claim listing APIs
- Support:
    - `page`
    - `size`

---

4. System Workflow

1. Admin creates users and assigns managers
2. Employee submits a claim
3. System assigns claim to Manager/Admin
4. Reviewer takes action (approve/reject)
5. Employee views updated status

---

5. Non-Functional Requirements

5.1 Performance

- System should handle standard CRUD operations efficiently
- Pagination should be used for large datasets

5.2 Logging

The system shall:

- Log incoming API requests
- Log key actions:
    - Claim submission
    - Approval / rejection
- Log errors

5.3 Validation & Error Handling

The system shall:

- Validate inputs before processing
- Validate email format and domain
- Return meaningful error responses
- Use centralized exception handling

5.4 Testing (Jacoco)

- Unit tests shall be written for core business logic.
- The system shall achieve at least **70% code coverage**.

5.5 Code Quality (Checkstyle)

- Follow a consistent coding standard using Checkstyle
- Ensure proper naming conventions for classes, methods, and variables
- Maintain proper code formatting (indentation, spacing, line length)
- Avoid unused imports and variables
- Ensure meaningful comments and JavaDocs where required
- Resolve all Checkstyle warnings before code merge

---

6. Must-Have Features

- User roles and management
- Claim submission
- Approval workflow
- Claim tracking
- Email validation (domain-based)
- Password Encryption
- Pagination
- Logging

---

7. Good to Have (try to achieve as many as you can)

- Password reset functionality
- File upload (documents/images)
- Resubmission of Claims (If Rejected)
- Editing of Claims (If not Approved yet)
- Ability to reply to comments received on claims (Both ways)
- Advanced authentication
- Audit history
- Multi-level approvals
- Complex UI
- Plugins(pmd)
- Spotbugs

---

8. Success Criteria

- Users can be created and assigned roles
- Claims can be submitted successfully
- Claims can be approved/rejected
- Status updates correctly
- Pagination works as expected
- Logs are generated for key actions